

Overview of Adagrad optimizer

Subject: Adagrad optimizer

1.

A compromise between computing the true gradient and the gradient at a single sample is to compute the gradient against more than one training sample (called a "mini-batch") at each step. This can perform significantly better than "true" stochastic gradient descent described, because the code can make use of vectorization libraries rather than computing each step separately as was first shown in where it was called "the bunch-mode back-propagation algorithm". It may also result in smoother convergence, as the gradient computed at each step is averaged over more training samples. The convergence of stochastic gradient descent has been analyzed using the theories of convex minimization and of stochastic approximation. Briefly, when the learning rates η decrease with an appropriate rate, and subject to relatively mild assumptions, stochastic gradient descent converges almost surely to a global minimum when the objective function is convex or pseudoconvex, and otherwise converges almost surely to a local minimum. This is in fact a consequence of the Robbins–Siegmund theorem.

2.

Implicit updates (ISGD) As mentioned earlier, classical stochastic gradient descent is generally sensitive to learning rate η . Fast convergence requires large learning rates but this may induce numerical instability. The problem can be largely solved by considering implicit updates whereby the stochastic gradient is evaluated at the next iterate rather than the current one:

3.

Backtracking line search Backtracking line search is another variant of gradient descent. All of the below are sourced from the mentioned link. It is based on a condition known as the Armijo–Goldstein condition. Both methods allow learning rates to change at each iteration; however, the manner of the change is different. Backtracking line search uses function evaluations to check Armijo's condition, and in principle the loop in the algorithm for determining the learning rates can be long and unknown in advance. Adaptive SGD does not need a loop in determining learning rates. On the other hand, adaptive SGD does not guarantee the "descent property" – which Backtracking line search enjoys – which is that $f(x_{n+1}) \leq f(x_n)$ for all n . If the gradient of the cost function is globally Lipschitz continuous, with Lipschitz constant L , and learning rate is chosen of the order $1/L$, then the standard version of SGD is a special case of backtracking line search.

4.

where E denotes taking the expectation with respect to the random choice of indices in the stochastic gradient descent scheme. Since this approximation does not capture the random fluctuations around the mean behavior of stochastic gradient descent solutions to stochastic differential equations (SDEs) have been proposed as limiting objects. More precisely, the solution to the SDE

5.

Approximations in continuous time For small learning rate η stochastic gradient descent $(w_n)_{n \in \mathbb{N}_0}$ can be viewed as a discretization of the gradient flow ODE

6.

RMSProp RMSProp (for Root Mean Square Propagation) is a method invented in 2012 by James Martens and Ilya Sutskever, at the time both PhD students in Geoffrey Hinton's group, in which the learning rate is, like in Adagrad, adapted for each of the parameters. The idea is to divide the learning rate for a weight by a running average of the magnitudes of recent gradients for that weight. Unusually, it was not published in an article but merely described in a Coursera lecture.

7.

This equation is implicit since w^{new} appears on both sides of the equation. It is a stochastic form of the proximal gradient method since the update can also be written as:

8.

$$m_w(t) := \beta_1 m_w(t-1) + (1 - \beta_1) \nabla_w L(t-1) \quad \{\displaystyle m_w(t) := \beta_1 m_w(t-1) + (1 - \beta_1) \nabla_w L(t-1)\}$$

9.

$$w^{\text{new}} := \arg \min_w \{ Q_i(w) + \frac{1}{2\eta} \|w - w^{\text{old}}\|^2 \} \quad \{\displaystyle w^{\text{new}} := \arg \min_w \left\{ Q_i(w) + \frac{1}{2\eta} \|w - w^{\text{old}}\|^2 \right\}\}$$

10.

As the algorithm sweeps through the training set, it performs the above update for each training sample. Several passes can be made over the training set until the algorithm converges. If this is done, the data can be shuffled for each pass to prevent cycles. Typical implementations may use an adaptive learning rate so that the algorithm converges. In pseudocode, stochastic gradient descent can be presented as :

11.

$$v_w(t) := \beta_2 v_w(t-1) + (1 - \beta_2) (\nabla_w L(t-1))^2 \quad \{\displaystyle v_w(t) := \beta_2 v_w(t-1) + (1 - \beta_2) (\nabla_w L(t-1))^2\}$$

12.

Notable applications Stochastic gradient descent is a popular algorithm for training a wide range of models in machine learning, including (linear) support vector machines, logistic regression (see, e.g., Vowpal Wabbit) and graphical models. When combined with the backpropagation algorithm, it is the de facto standard algorithm for training artificial neural networks. Its use has been also reported in the Geophysics community, specifically to applications of Full Waveform Inversion (FWI). Stochastic gradient descent competes with the L-BFGS algorithm, which is also widely used. Stochastic gradient descent has been used since at least 1960 for training linear regression models, originally under the name ADALINE. Another stochastic gradient descent algorithm is the least mean squares (LMS)

adaptive filter.

13.

Sign-based stochastic gradient descent Even though sign-based optimization goes back to the aforementioned Rprop, in 2018 researchers tried to simplify Adam by removing the magnitude of the stochastic gradient from being taken into account and only considering its sign. This results in a significantly lower communication cost of transferring gradients from workers to the parameter server. In this sense, it serves to better compress the gradient information, while having comparable convergence to standard SGD.

14.

for $\Sigma(w) = \frac{1}{n^2} \left(\sum_{i=1}^n Q_i(w) - Q(w) \right) \left(\sum_{i=1}^n Q_i(w) - Q(w) \right)^T$
$$\Sigma(w) = \frac{1}{n^2} \left(\sum_{i=1}^n Q_i(w) - Q(w) \right) \left(\sum_{i=1}^n Q_i(w) - Q(w) \right)^T$$
 where dB_t denotes the Ito-integral with respect to a Brownian motion is a more precise approximation in the sense that there exists a constant $C > 0$ such that